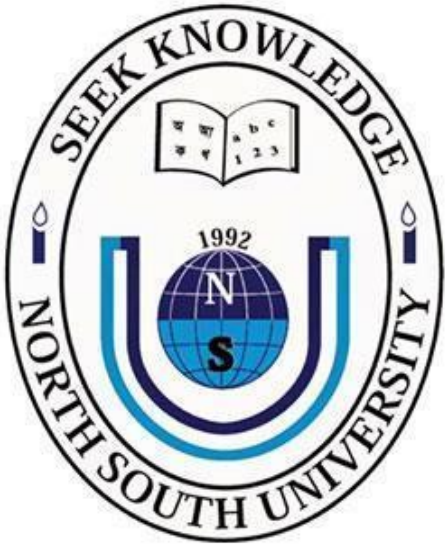


DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

PROJECT REPORT

Submitted by:

Group: 7

Name: Eusra Amreen

Id: 2321505642

Email:eusra.amreen.232@northsouth.edu

Name: Zahin Al Mahmud

Id: 2323770642

Email:zahin.mahmud.232@northsouth.edu

Course: CSE 215L

Section: 10

Submitted to: Md. Abdullah Al Sayed, Lab Instructor
Muhammad Shafayet Osman, Lecturer, North South University

CONTENTS

CONTENTS 2

ATM MANAGEMENT SYSTEM 3

- **INTRODUCTION TO ATM** 3
- **INTRODUCTION TO ATM MANAGEMENT SYSTEM** 3
- **MAIN GOAL** 3
- **IMPORTANCE** 3
- **FUNCTIONALITIES** 4
- **UML** 5
- **FEATURES CHART** 6
- **VISUALIZATION** 13
- **LIBRARIES** 23
- **CHALLENGES** 24
- **FUTURE ADVANCEMENTS** 25
- **LESSONS LEARNT** 25
- **REFERENCE** 26

ATM MANAGEMENT SYSTEM

INTRODUCTION TO ATM

Automated Teller Machines, which are more popularly known as ATMs, are a way of performing quick and easy transactions on an individual basis. These machines allow you to undergo self-serviced transactions without the help of an actual teller and without having to visit the bank branch. Automated teller machines or ATMs allow many transactions, including cash withdrawals, cash deposits, and fund transfers.[1]

INTRODUCTION TO ATM MANAGEMENT SYSTEM

This project presents the development of an ATM management system implemented in Java, aimed at enhancing the efficiency and security of banking operations.

- ➔ The project adopts modular design principles to ensure scalability and ease of maintenance.
- ➔ Overall, the Java-based ATM management system provides a comprehensive solution to optimize banking processes and enhance customer satisfaction.

MAIN GOAL

ATMs currently available have limited features like withdrawing money so we decided to **add more features** like depositing and transferring money and registration of accounts as well for the **convenience of the customers.**

IMPORTANCE

- ❖ Allows the customer to **undergo self-serviced transactions** without the help of an actual teller and without having to visit the bank branch.[1]
- ❖ This ATM allows customers to enjoy **more privileges** compared to other ATMs.
- ❖ Allows the admin to manage and oversee the ATM's operation better.
- ❖ Highly convenient and accessible for anyone.

FUNCTIONALITIES

1. **User authentication and encryption** :
   - Validate users through login credentials (e.g., username and password)
   - Implement encryption techniques to secure sensitive data such as PINs and account information.

2. **Account balance inquiry:** Allow users to check their account balance.

3. **Cash withdrawal**: Enable users to withdraw cash from their accounts.

4. **Funds transfer**: Allow users to transfer funds between their accounts or to other accounts.

5. **Mini statement generation:** Provide users with a summary of recent transactions.

6. **PIN management**: Allow users to change their PINs for security purposes.

7. **Account management**: Allow users to open new accounts, close accounts, or update account information.

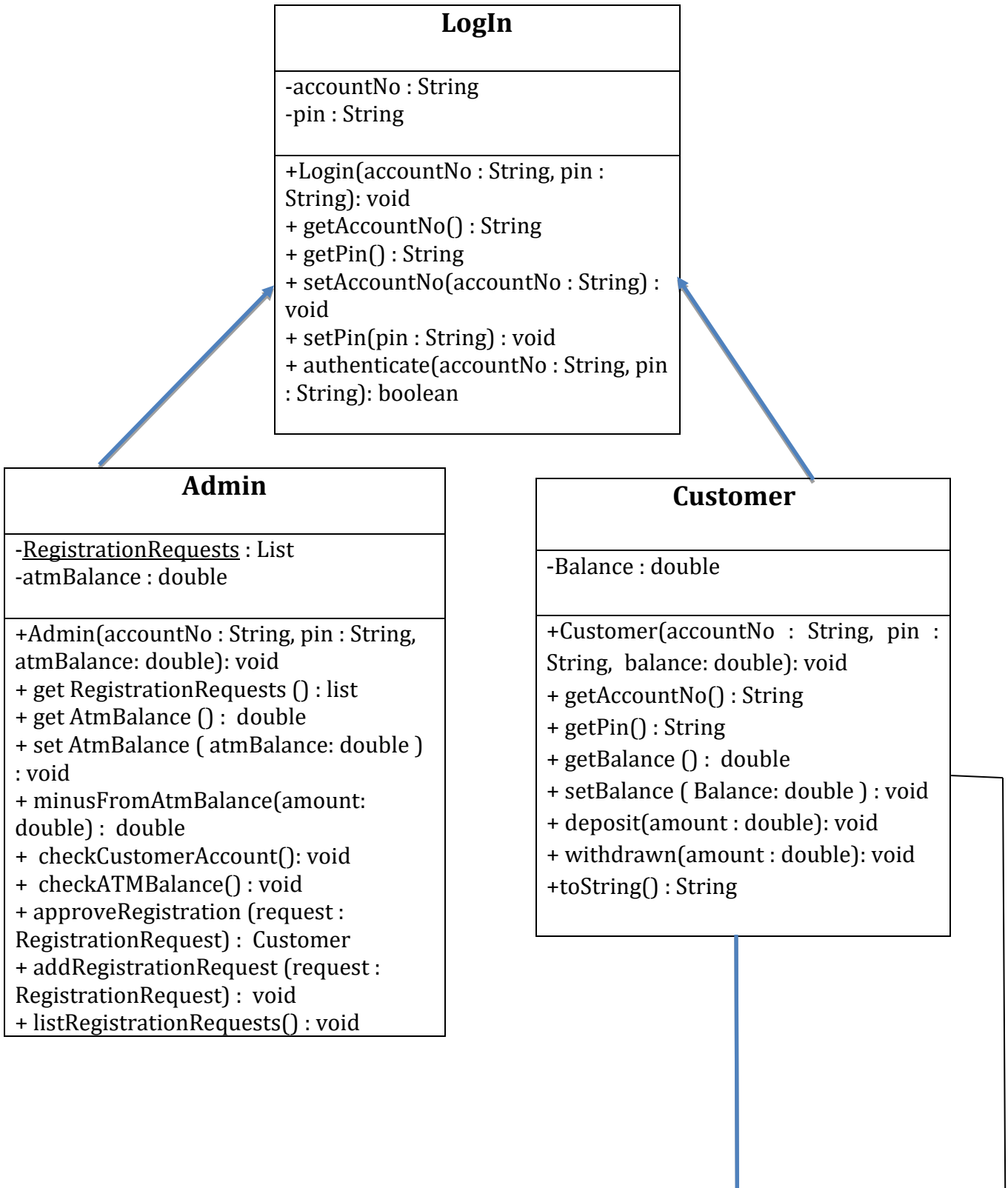
8. **Transaction receipts**: Generate transaction receipts for users' records.

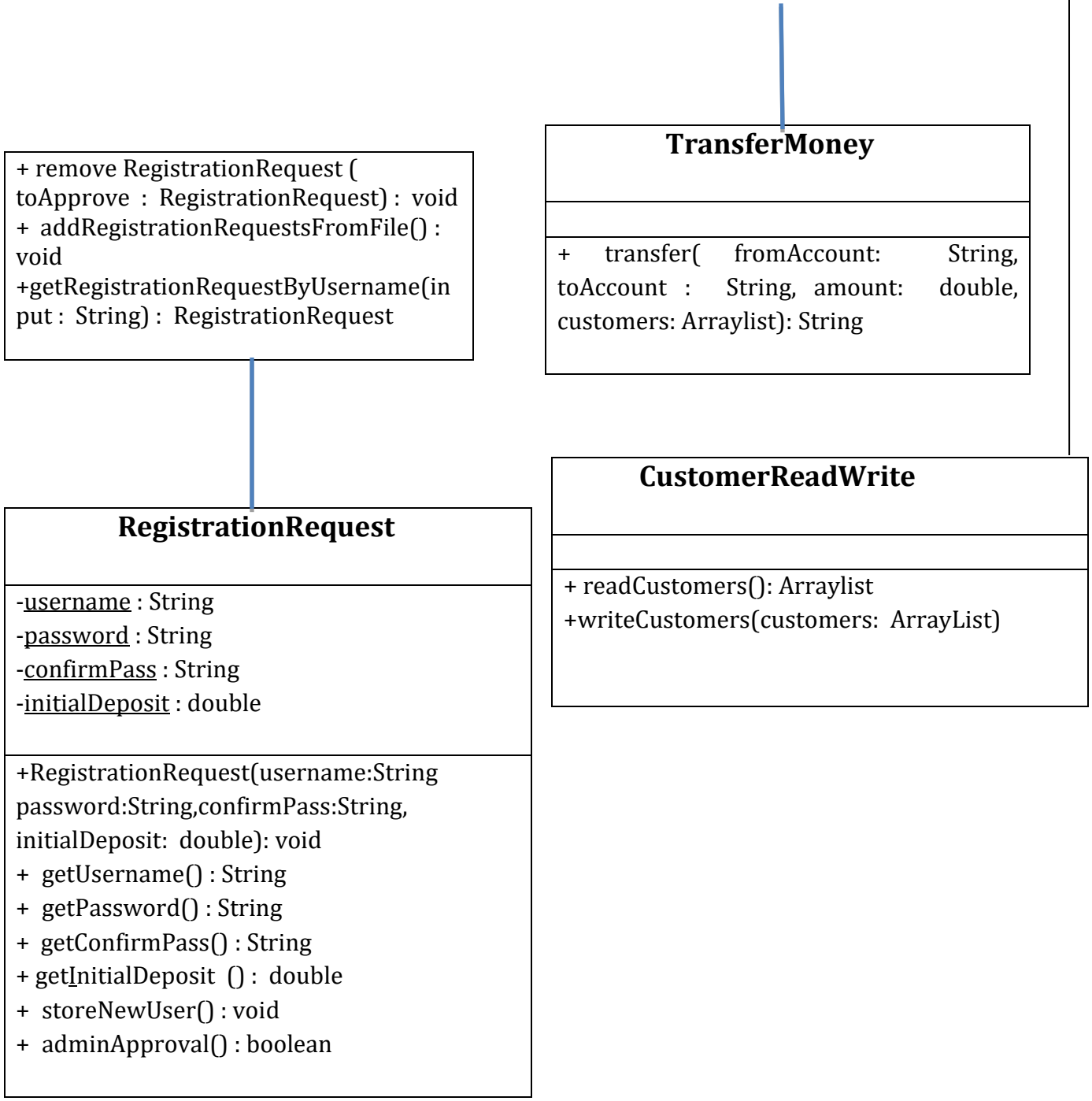
9. **Administrator functionalities**: Provide administrators with privileges to manage user accounts, monitor transactions, and generate reports.

10. **Error handling**: Implement robust error handling mechanisms to handle exceptions and errors gracefully.

11. **User interface(GUI):** Design a user-friendly interface for both customers and administrators, with intuitive navigation and clear instructions.

UML





FEATURES CHART

Classes :	Purpose :	System Manual:	Contributor:
ATM (Main)	Implementation of all classes and methods at main. *(N.B: The only runnable*	After running, you will get a welcome notice and will see a main menu with multiple options, Log in Credentials : *(used for trial*	Eusra

	class due to the main)	*before implementing file)* **Admin**: admin123, password **Customer**: Amreen, 5001 ❖ **For option 1**: You will be directed to the admin log in page where you can input your account id and pin and log in if you are an admin to oversee the customer's view permitted details, atm balance and approve new customer requests implementing the ***adminLogin*** () and ***approveRegistrationRequests ()*** methods by using **Admin** class. ❖ **For option 2**: You will be directed to the customer log in page where you have to input your existing account id and pin to access the portal to withdraw or deposit or transfer mone or view account details implementing the ***CustomerLogin***() method by using **Customer, Transfer and CustomerReadWrite** class. ❖ **For option 3**: You will be directed to the new customer registration page where you can you have to input an account id, pin and initial deposit amount to register implementing the ***Registration()*** method by using **RegistrationRequest** class. ❖ **For option 4**: You exit the system and program will be terminated.	

LogIn	Login in to the system	Attributes: ❖ **accountNo** : a String value ❖ **Pin** : a String value Methods : ❖ **Constructor: LogIn(***String accountNo, String pin***):** Serves its purpose at main while implementing the class by taking the values of the mentioned attributes and returning void. ❖ **Getters**: **getAccountNo(), getPin():** Serves a purpose to return a String value set upon it. ❖ **Setters**: **setAccountNo(***String accountNo***), setPin(***String pin***):** Serves a purpose to take a String value from the user and return void. ❖ **Behavior**: **authenticate(***String accountNo, String pin***)**: Serves a purpose to authenticate the account and password while returning a boolean value(true or false). This class is implemented whenever log in occurs in the system.	Eusra
Customer	Customer's functions in the system *(N.B: Inherits LogIn)*	Attributes: ❖ **balance** : a double value Methods : ❖ **Constructor: Customer(***String accountNo, String pin, double balance***):** Serves its purpose at main while implementing the class by taking the values of the mentioned and parent class's attributes and returning void.	Eusra

		❖ **<u>Getters</u>**: **getBalance():** Serves a purpose to return a double value set upon it. ❖ **<u>Setters</u>**: **setBalance(***double balance***):** Serves a purpose to take a double value from the user and return void. ❖ **<u>Behavior</u>**: **deposit(***double amount***) :** takes the double value of amount to deposit from the customer and calculates the new balance atm and customer and prints it. It has a void return type for convenience. **withdraw(***double amount***):** takes the double value of amount to withdraw from the customer and calculates the new balance of both atm and customer and prints it. It has a void return type for convenience. **toString():** returns the balance in string format. This class is implemented during the customer session in switch case.	
Admin	Admin's functions in the system *(N.B: Inherits LogIn)*	Attributes: ❖ **atmBalance** : a double value ❖ **RegistrationRequest**: a final Arraylist variable Methods : ❖ **<u>Constructor:</u> Customer(***String accountNo, String pin, double atmBalance***):** Serves its purpose at main while implementing the class by taking the values of the mentioned and parent class's	Eusra

		attributes and returning void. ❖ **<u>Getters</u>**: **getAtmBalance():** Serves a purpose to return a double value set upon it. ❖ **getRegistrationRequest():** Serves a purpose to return a list value set upon it. ❖ **<u>Setters</u>**: **setAtmBalance(***double atmBalance***):** Serves a purpose to take a double value from the user and return void. ❖ **<u>Behavior</u>**: **listRegistrationRequest():** Serves a purpose to view registration requests. **checkAtmBalance():** Serves a purpose to view atm balance. **addToAtmBalance():** Adds the amounts deposited to the atm balance. **minusFromAtmBalance():** Subtracts the amounts withdrawn from the atm balance. **checkCustomerAccount ():** Previews Customer accounts with balance. **checkATMBalance ():** Previews atm's balance **approveRegistration ():** Approves new customer requests. **addRegistrationRequest ():** Adds new customer requests to	

		the file. **listRegistrationRequests ():** Previews new customer requests. **removeRegistrationRequest ():** Removes the approved new customer's requests from the requests file. **addRegistrationRequestsFromFile ():** Adds new customer requests to the system from file. **getRegistrationRequestByUsername ():** Finds the request to approve by username a.k.a. account id.	
TransferMoney	Transferring of money and adding and writing in files	❖ **Behavior**: **transfer(***String fromAccount, String toAccount, double amount, ArrayList<Customer> customers***)** : Takes in the user's account id and another user's id and an amount then transfers money from one user to another user and changes their balance and updates in the file and returns a mini statement.	Zahin
RegistrationRequest	Registration of new customers in the system	Attributes: ❖ **private final String username;** ❖ **private final String password, confirmPass;** ❖ **private final double initialDeposit** Methods : ❖ **Constructor: RegistrationRequest(***String*	Eusra & Zahin

		*username, String password, String confirmPass, double initialDeposit***):** Serves its purpose at main while implementing the class by taking the values of the mentioned attributes and returning void. ❖ **<u>Getters</u>**: **getUsername (),getPassword(),getInitialDeposit():** Serves a purpose to return a String value set upon it. ❖ **<u>Setters</u>**: No Setters available because every variable is final. ❖ **<u>Behavior</u>**: **storeNewUser()**: Serves a purpose to to store new users in file and returns void. **adminApproval()**: Reading and writing in file after admin approval and returns a boolean value. This class is implemented whenever registration and admin's approval of requests occurs in the system.	
CustomerReadWrite	Reads and writes customers information in a csv file.	❖ **<u>Behavior</u>**: **readCustomers ()**: Reads the customers' data from file. **writeCustomers**: Writes customers' data in a csv file.	Zahin

VISUALIZATION

- Login:

```java
public class LogIn {
    4 usages
    private String accountNo;
    4 usages
    private String pin;
    no usages
    public LogIn(){

    }
    2 usages
    public LogIn(String accountNo, String pin){
        this.accountNo = accountNo;
        this.pin = pin;
    }
    8 usages
    public String getAccountNo() { return accountNo ; }

    1 usage
    public String getPin() { return pin; }

    no usages
    public void setAccountNo(String accountNo) { this.accountNo = accountNo; }

    no usages
    public void setPin(String pin) { this.pin = pin; }

    2 usages
    public boolean authenticate(String accountNo, String pin) {
        return this.accountNo.equals(accountNo) && this.pin.equals(pin);
    }
}
```

Output:

```
Enter admin username: admin123
Enter admin password: password
```

```
Enter customer username: Amreen
Enter customer password: 5001
```

➢ Customer :

```java
import java.sql.Timestamp;
16 usages
public class Customer extends LogIn{
    9 usages
    private double balance;

    5 usages
    public double getBalance() { return balance; }

    2 usages
    public void setBalance(double balance) { this.balance = balance; }

    2 usages
    public Customer(String accountNo, String pin, double balance) {
        super(accountNo, pin);
        this.balance = balance;
    }

    1 usage
    public void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited: " + amount + " Taka.  New balance: " + balance +" Taka "+ Timestamp.valueOf( s: " 2024-06-09 13:22:15"));

    }

    1 usage
    public void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount + " Taka. Remaining balance: " + balance + " Taka " + Timestamp.valueOf( s: " 2024-06-09 13:22:15"));
        } else {
            System.out.println("Insufficient funds.");
        }
    }


    @Override
    public String toString() { return "Customer balance = " + balance + " Taka "; }
}
```

This Customer class gives you the following output after intaking user's choices.

Output:

```
Welcome to the ATM Management System!!
1. Admin Login
2. Customer Login
3. Customer Registration
4. Exit
Please enter your choice: 2
Enter customer username: Amreen
Enter customer password: 5001
Customer Menu:
1. Withdraw Money
2. Deposit Money
3. Transfer Money
4. View Balance
5. Log out
Please enter your choice: 1
Input the amount to withdraw :
300
Withdrawn: 300.0 Taka. Remaining balance: 6500.0 Taka 2024-06-09 13:22:15.0

Customer Menu:
1. Withdraw Money
2. Deposit Money
3. Transfer Money
4. View Balance
5. Log out
Please enter your choice: 2
Input the amount to deposit :
400
Deposited: 400.0 Taka.  New balance: 6900.0 Taka 2024-06-09 13:22:15.0
```

- Admin

```
public class Admin extends LogIn{

    10 usages
    private final List<RegistrationRequest> RegistrationRequests;
    8 usages
    private double atmBalance; // Assuming there's a field to track ATM balance

    1 usage
    public Admin(String username, String password, double atmBalance) {
        super(username, password);
        this.RegistrationRequests = new ArrayList<>();
        this.atmBalance = atmBalance;
    }

    no usages
    public List<RegistrationRequest> getRegistrationRequests() { return RegistrationRequests; }

    no usages
    public double getAtmBalance() { return atmBalance; }

    no usages
    public void setAtmBalance(double atmBalance) { this.atmBalance = atmBalance; }
    1 usage
    public double addToAtmBalance(double amount){
        atmBalance =+amount;
        return atmBalance;
    }
    1 usage
    public double minusFromAtmBalance(double amount){
        atmBalance -= amount;
        return atmBalance;
    }
    1 usage
    public void checkCustomerAccount() {
        ArrayList<Customer> customers = new ArrayList<>();
        CustomerReadWrite crw = new CustomerReadWrite();
        customers = crw.readCustomers(); // Reads all the customers from users.csv
        for(Customer c : customers) {
            System.out.println("Username: " + c.getAccountNo() + ", Balance " + c.getBalance());
        }
```

```java
public void checkATMBalance() {
    System.out.println("The current ATM balance is: " + atmBalance + " Taka " +Timestamp.valueOf( s: "  2024-06-09 13:22:15") );
}


1 usage
public Customer approveRegistration(RegistrationRequest request) {
    // Add logic to save the new customer to the database or file system
    request.adminApproval();
    return new Customer(request.getUsername(), request.getPassword(), request.getInitialDeposit());
}

1 usage
public void addRegistrationRequest(RegistrationRequest request) {
    RegistrationRequests.add(request);
    request.storeNewUser();
}

1 usage
public void listRegistrationRequests() {
    addRegistrationRequestsFromFile();
    for (RegistrationRequest request : RegistrationRequests) {
        System.out.println("Username: " + request.getUsername() + ", Initial Deposit " + request.getInitialDeposit());
    }
}

1 usage
public void removeRegistrationRequest(RegistrationRequest toApprove) {
    for(int i=0; i<RegistrationRequests.size(); i++) {
        if(RegistrationRequests.get(i).getUsername() == toApprove.getUsername()) {
            RegistrationRequests.remove(i);
        }
    }
}
```

```java
    1 usage
    public void addRegistrationRequestsFromFile() {
        RegistrationRequests.clear();
        File fr = new File( pathname: "new-user.csv");
        try {
            BufferedReader reader = new BufferedReader(new FileReader(fr));
            String line;
            while ((line = reader.readLine()) != null) {
                String[] user = line.split( regex: ",");
                String username = user[0];
                String pass = user[1];
                double balance = Double.parseDouble(user[2]);
                RegistrationRequest rr = new RegistrationRequest(username, pass, pass, balance);
                RegistrationRequests.add(rr);
            }
            reader.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    1 usage
    public RegistrationRequest getRegistrationRequestByUsername(String input) {

        for (RegistrationRequest request : RegistrationRequests) {
            if(request.getUsername().equalsIgnoreCase(input)) {
                return request;
            }
        }
        return null;
    }
}
```

This Admin class gives you the following output after intaking user's choices which is evident after implemention of main.

Output:

```
Welcome to the ATM Management System!!
1. Admin Login
2. Customer Login
3. Customer Registration
4. Exit
Please enter your choice: 1
Enter admin username: admin123
Enter admin password: password
Admin Menu:
1. View Customer Account Details
2. Check ATM Balance
3. Approve Registration
4. Logout
Please enter your choice: 1
Username: Mahir, Balance 21300.0
Username: Bob, Balance 4000.0
Username: Roy, Balance 1500.0
Username: Amreen, Balance 6500.0

Admin Menu:
1. View Customer Account Details
2. Check ATM Balance
3. Approve Registration
4. Logout
Please enter your choice: 2
The current ATM balance is: 100000.0 Taka 2024-06-09 13:22:15.0

Admin Menu:
1. View Customer Account Details
2. Check ATM Balance
3. Approve Registration
4. Logout
Please enter your choice: 3
Username: Jack, Initial Deposit 10000.0
Enter the username of the customer to approve or type 'cancel' to return:
cancel
```

- TransferMoney

```java
public class TransferMoney {

    1 usage
    public String transfer(String fromAccount, String toAccount, double amount, ArrayList<Customer> customers) {
        int Count=0;
        for(Customer c : customers) {
            if(c.getAccountNo().equals(fromAccount)) {
                if(c.getBalance() >= amount) Count++;
            }
            if(c.getAccountNo().equals(toAccount)) {
                Count++;
            }
        }
        if(Count != 2) {
            return "Coudn't transfer money";
        }
        for(Customer c : customers) {
            if(c.getAccountNo().equals(fromAccount)) {
                c.setBalance(c.getBalance()-amount);
            }
            if(c.getAccountNo().equals(toAccount)) {
                c.setBalance(c.getBalance()+amount);
            }
        }
        CustomerReadWrite crw = new CustomerReadWrite();
        crw.writeCustomers(customers);
        return "Money Transferred Sucessfully.";
    }

}
```

Output:

```
Customer Menu:
1. Withdraw Money
2. Deposit Money
3. Transfer Money
4. View Balance
5. Log out
Please enter your choice: 3
Input the id to transfer money to:
Mahir
Input the amount to transfer money to:
400
Money Transferred Sucessfully.
```

- RegistrationRequest

```
public class RegistrationRequest {
    4 usages
    private final String username;
    4 usages
    private final String password, confirmPass;
    3 usages
    private final double initialDeposit;

    2 usages
    public RegistrationRequest(String username, String password, String confirmPass, double initialDeposit) {
        this.username = username;
        this.password = password;
        this.confirmPass = confirmPass;
        this.initialDeposit = initialDeposit;
    }

    // Getters
    5 usages
    public String getUsername() { return username; }

    1 usage
    public String getPassword() { return password; }

    2 usages
    public double getInitialDeposit() { return initialDeposit; }


    1 usage
    public void storeNewUser() {
        if (password.equals(confirmPass)) {
            try (BufferedWriter writer = new BufferedWriter(new FileWriter( fileName: "new-user.csv",  append: true))) {

                writer.write( str: username + "," + password + "," + initialDeposit);
                writer.newLine();
                writer.close();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
```

```
    public boolean adminApproval() {

        boolean changed = false;
        File fr = new File( pathname: "new-user.csv");
        File frtemp = new File( pathname: "temp-new-user.csv");
        File fw = new File( pathname: "users.csv");
        try {
            BufferedReader reader = new BufferedReader(new FileReader(fr));
            BufferedWriter writer = new BufferedWriter(new FileWriter(fw,  append: true));
            BufferedWriter writerTemp = new BufferedWriter(new FileWriter(frtemp));

            String line;
            while ((line = reader.readLine()) != null) {
                String[] user = line.split( regex: ",");
                if (user[0].equals(username)) {
                    writer.write(line);
                    writer.newLine();
                    changed = true;
                } else {
                    writerTemp.write( str: user[0] + "," + user[1] + "," + user[2]);
                    writerTemp.newLine();
                }
            }
            reader.close();
            writer.close();
            writerTemp.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
        if (changed) {
            fr.delete();
            frtemp.renameTo(fr);
            return true;
        } else {
            frtemp.delete();
        }
        return false;


    }
}
```

This class gives you the following output after intaking user’s choices.

Output:

```
Welcome to the ATM Management System!!
1. Admin Login
2. Customer Login
3. Customer Registration
4. Exit
Please enter your choice: 3
Enter name :
Zahin
Enter pin :
4001
Enter pin again to confirm :
4001
Enter initial deposit :
2000
Your registration request has been submitted for admin approval.
```

- CustomerReadWrite

```
    public ArrayList<Customer> readCustomers() {
        ArrayList<Customer> customers = new ArrayList<>();
        File fr = new File( pathname: "users.csv");  // adding customers from file
        try {
            BufferedReader reader = new BufferedReader(new FileReader(fr));
            String line;
            while ((line = reader.readLine()) != null) {
                String[] user = line.split( regex: ",");
                String username = user[0];
                String pass = user[1];
                double balance = Double.parseDouble(user[2]);
                Customer cus = new Customer(username, pass, balance);
                customers.add(cus);
            }
            reader.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
        return customers;
    }

    3 usages
    public void writeCustomers(ArrayList<Customer> customers) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter( fileName: "users.csv"))) {
            for(Customer c : customers) {
                writer.write( str: c.getAccountNo() + "," + c.getPin() + "," + c.getBalance());
                writer.newLine();
            }
            writer.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

}
```

This class gives you the following output after implementing files.

Output(File) :

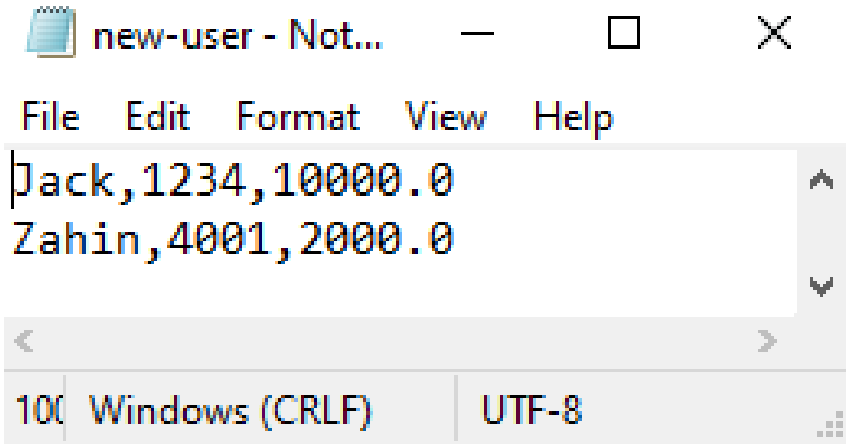


Figure : Registration requests

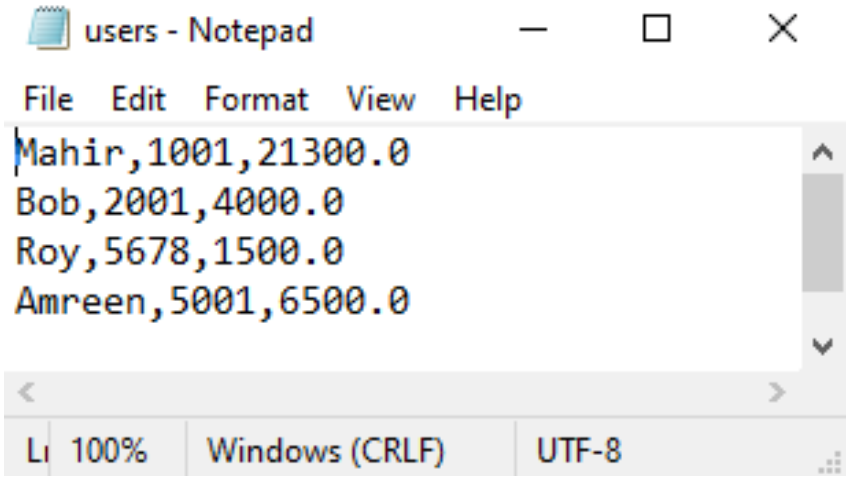


Figure : Existing customer

LIBRARIES

1. **ArrayList**: An ArrayList is a dynamic array that allows you to store and manipulate a collection of objects. It provides methods for adding, removing, and

accessing elements. Use it when you need a resizable list with constant-time access to elements by index.

2. **Scanner**: The Scanner class is used for parsing tokens from input streams (such as standard input or files). It's commonly used for reading user input or reading data from files. You can use it to read various data types (e.g., integers, doubles, strings) from the input stream.

3. **BufferedReader**: A BufferedReader reads text from a character-based input stream (e.g., a file). It's more efficient than reading directly from the stream because it buffers the data, reducing the number of system calls. Use it for reading large files line by line.[3]

4. **File**: The File class represents a file or directory path. It provides methods for creating, deleting, and querying file properties. Use it when you need to work with files or directories in your program.

5. **FileReader**: A FileReader reads character data from a file. It's a simple way to read text files, but it lacks buffering. For better performance, consider using BufferedReader alongside FileReader.

6. **BufferedWriter**: A BufferedWriter writes text to an output stream (e.g., a file) efficiently by buffering the data. It's useful when you have multiple small writes, as it reduces the number of system calls and improves performance.[3]

7. **FileWriter**: A FileWriter writes character-oriented data to a file. It's similar to BufferedWriter, but without the buffering. Use it when you need to write text data to a file.[2]

CHALLENGES

- **Binary file reading system**: Fixed through trial and error.
- **Approving Registration**: I was facing difficulties due to improper implementation of the method containing this feature's mechanism which was fixed when properly implemented in the main.
- **GUI**: Even though I managed to create classes using Java swing, I wasn't able to implement them properly due to lack of experience and time.

- **Viewing Customer list** : At first I was trying to fill an array of customer objects manually and eventually was getting a blank output then print it but realized that it's better done with arraylist so learnt it and used it.
- **Creating Interface and abstract :** I tried to prepare abstract and interface for it but found them a little bit redundant for this project so eliminated them but might bring it back upon requirements in the future.

FUTURE ADVANCEMENTS

- GUI's (Graphical User Interface) proper implementation by using Java Fx or Java Swing.
- Implementation of a Biometric log-in system.
- Log all transactions for auditing and troubleshooting purposes.
- Improving the Admin's features like sorting view of customer IDs in different ways, and restricting existing customers due to security concerns.
- Further Error Handlings will be implemented.
- Bug fixes.

LESSONS LEARNT

- Object Oriented Programming :
- Inheritance : Applied in Customer, Admin
- Methods : Applied in every classes
- Overriding : Applied in Customer
- Error Handling : Applied in Admin, RegistrationRequest, CustomerReadWrite
- File : Applied in Admin, RegistrationRequest, CustomerReadWrite
- Use of .csv file for better visuals but it's restricted.
- Use of GUI by java swing.
- Use of final,object and arraylist variables
- Arraylist : Applied in Admin, RegistrationRequest

REFERENCE

1. Cleartax(2024) : Atm Automated Teller Machine - Definition, What is Atm Automated Teller Machine, Advantages of Atm Automated Teller Machine, and Latest News - ClearTax
2. GeeksforGeeks(2022) : FileWriter Class in Java - GeeksforGeeks
3. Owlcation(2023) : Java Examples: BufferedReader and BufferedWriter - Owlcation
4. W3Schools : Java Read Files (w3schools.com)

THANK YOU!

Crafted by Eusra Amreen
Reviewed by Zahin Al Mahmud